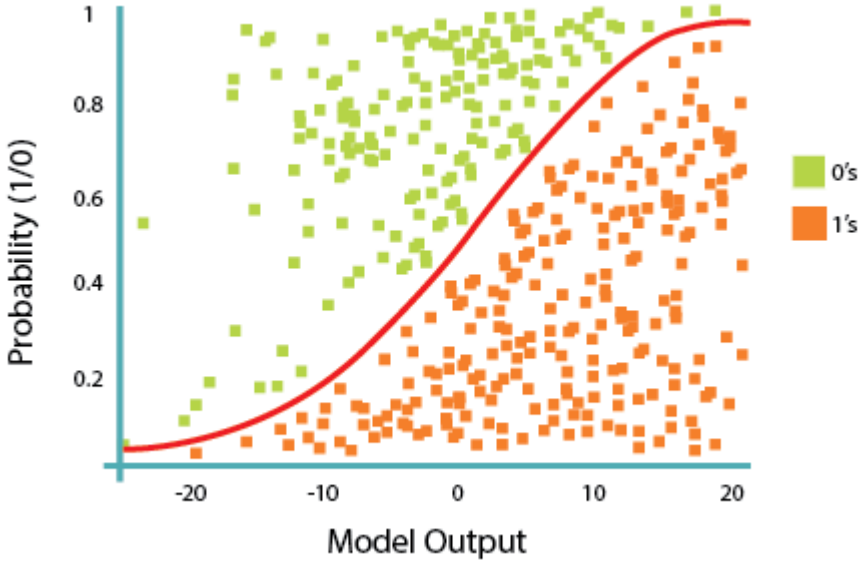
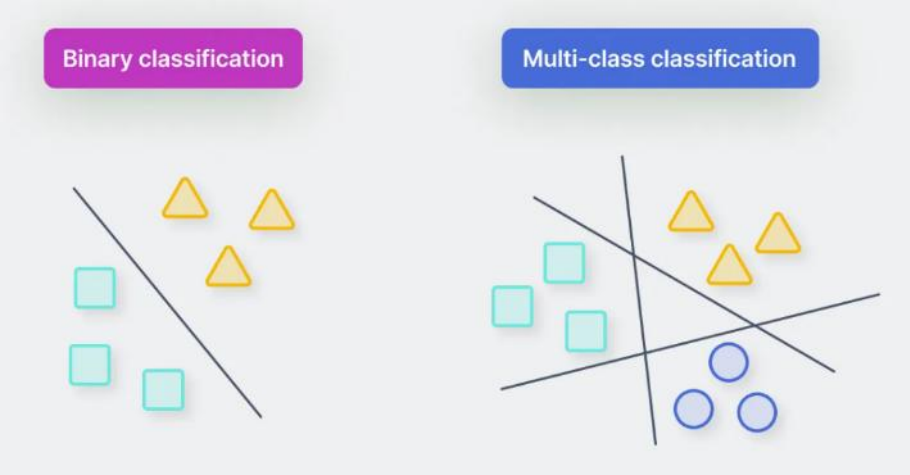
Binary Classification with Logistic Regression



Outline

- Classification
- Logistic Regression
- Classification Model Evaluation

Classification



Classification Models

🎯 **Goal:** predict the **class label** of an input using **supervised learning**

1 Training learn from examples

📘 **Labeled data**
features + known labels

➡

⚙️ **Learning algorithm**
finds patterns

➡

🤖 **Classifier**
the trained model

💡 The algorithm keeps adjusting the model parameters until its predictions match the known labels.

2 Inference predict on new data

📥 **New input**
label unknown ❓

➡

🤖 **Classifier**
the trained model

➡

🏷️ **Predicted label**
most likely class

💡 The trained model assigns the most likely class to data it has never seen.

Classification Examples

Same idea, different labels: **Binary** = 2 classes | **Multi-class** = 3 or more classes



Spam Detection

Is this email spam or not spam? Learns from text and sender data.

Binary



Sentiment Analysis

Is this review positive, negative or neutral?

Multi-class



Medical Diagnosis

Does the patient have the condition or not? Uses symptoms and history.

Binary



Credit Risk Assessment

Is the applicant low, medium or high risk? Uses score, income, debt.

Multi-class



Image Recognition

Is it a cat, dog or bird? Learns visual features from pixels.

Multi-class

Classification Algorithms



Logistic Regression

Estimates the probability of a class using an S-shaped curve.

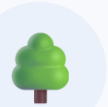
✅ **Best for:** Fast, interpretable baseline



K-Nearest Neighbors

Looks at the K closest examples and takes a vote.

✅ **Best for:** Small datasets, simple intuition



Decision Trees

Asks a series of yes/no questions to reach a decision.

✅ **Best for:** Easy-to-explain rules



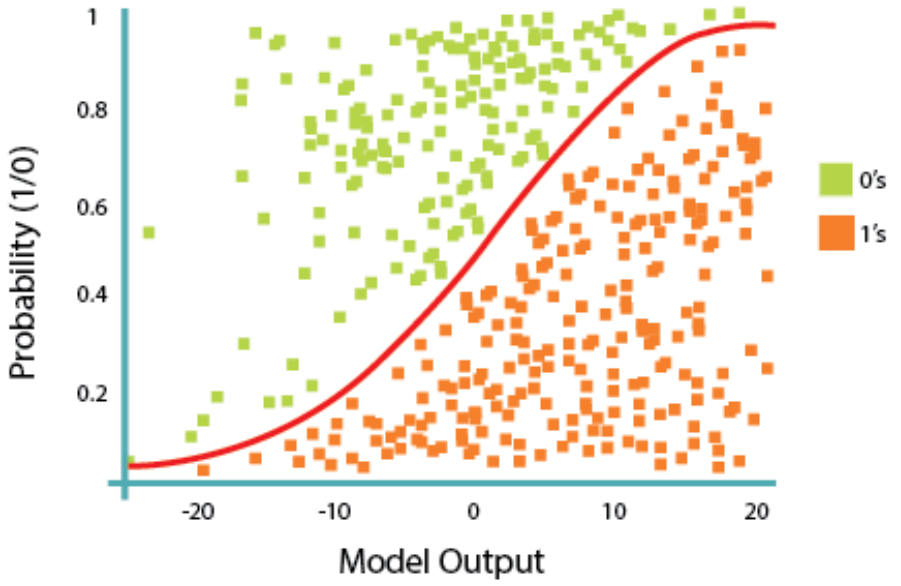
Random Forests

Many trees vote together for a more accurate, stable result.

✅ **Best for:** Higher accuracy on tabular data

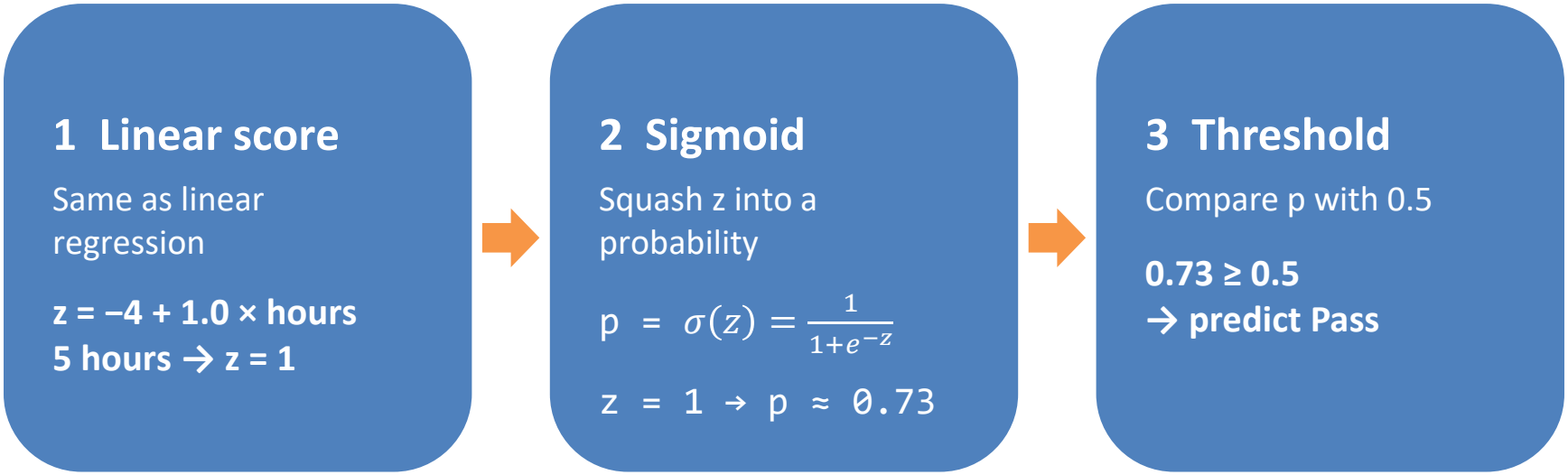
➕ Many others: SVM, Naive Bayes, Neural Networks 👉 **Next: Logistic Regression**

Logistic Regression

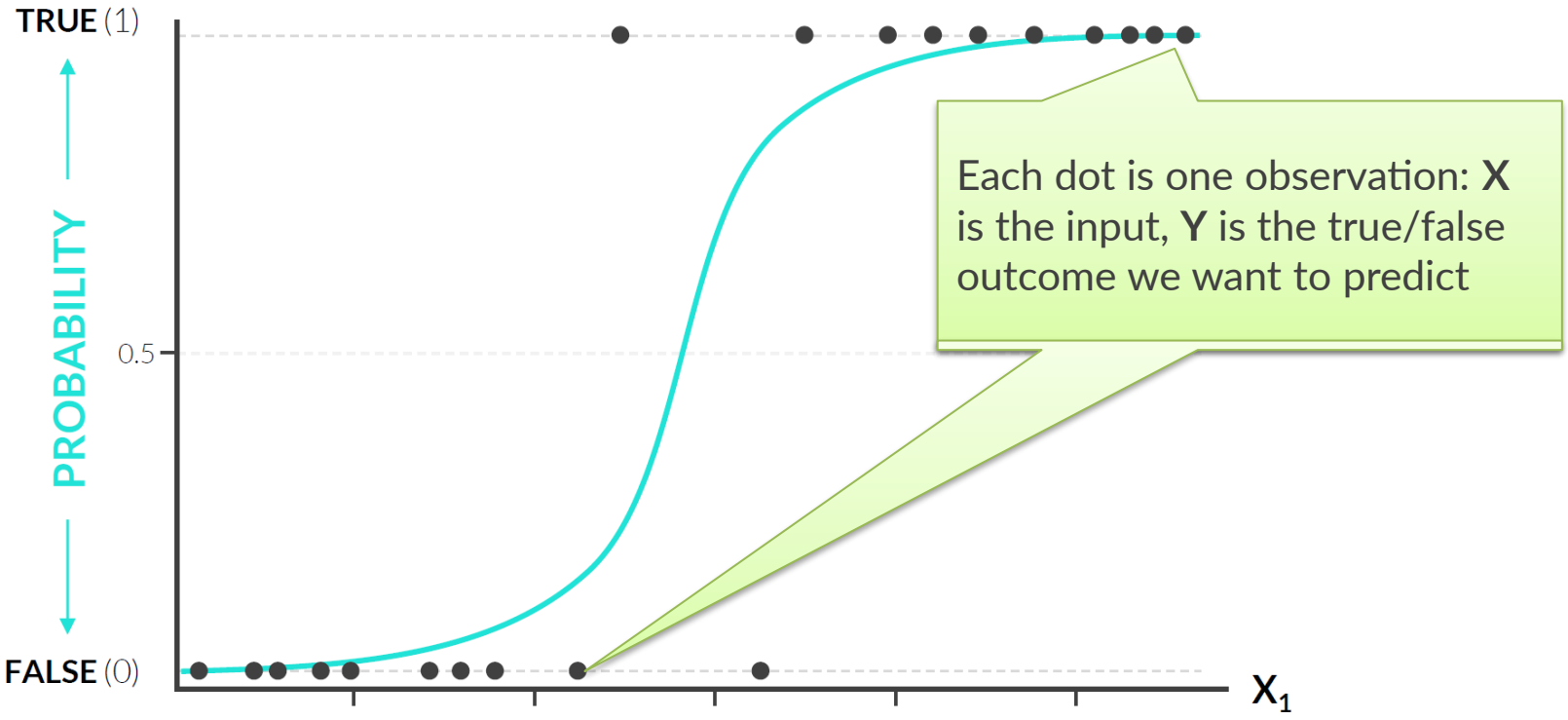


From Linear to Logistic Regression

- **Linear regression** predicts a number, e.g. an exam score
- **Logistic regression** reuses the same linear score, then turns it into a **probability (0 to 1)** for a yes/no answer
- **Example:** will a student pass, given hours studied?



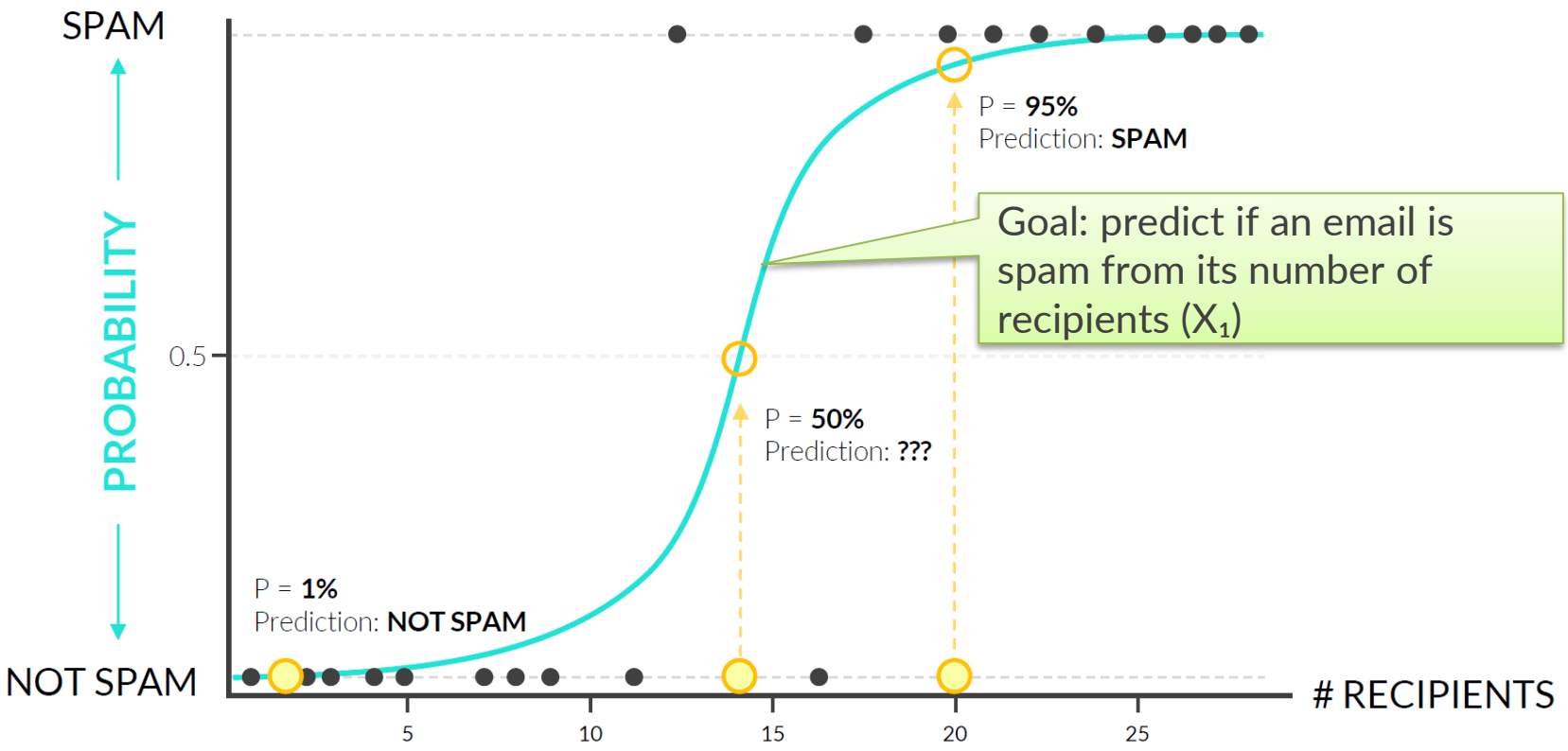
How the S-Curve Works



- Logistic regression learns an **S-shaped curve** bounded between 0 and 1.
- The curve shows the **probability** that **Y = TRUE** for a given input $\mathbf{X_1}$

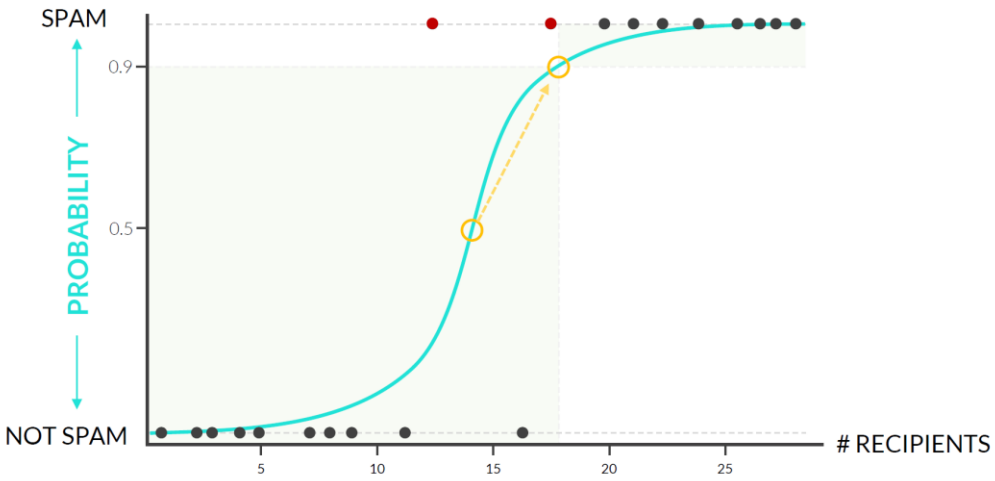
  Low X_1 → probability near 0 (FALSE). High X_1 → probability near 1 (TRUE).

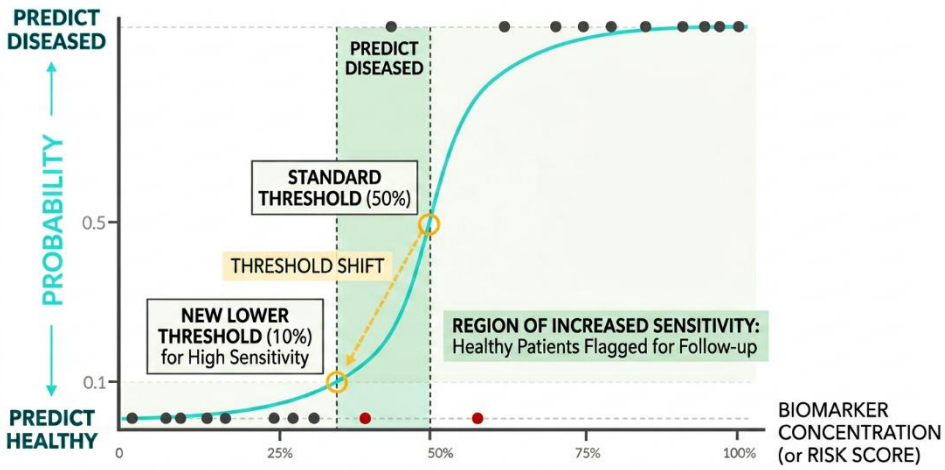
Example: Is This Email Spam?



- Read the curve: more recipients (X_1) → higher chance the email is spam
- Example: 2 recipients → P ≈ 1% → **Not spam**. 20 recipients → P ≈ 95% → **Spam**
- Rule of thumb: **P > 0.5 → predict Spam**, otherwise Not spam (0.5 is the common default threshold)

Is 50% Always the Right Threshold?





- The threshold is a **business decision**: it depends on which mistake costs more

Example 1: Spam filter

Costly mistake: **false positive** *(a false alarm: a real email lands in spam)*

✅ Use a **high threshold** (e.g., 90%): flag as spam only when very sure

Example 2: Cancer screening

Costly mistake: **false negative** = ***a miss***: *a sick patient is told they are healthy*

✅ Use a **low threshold** (e.g., 10–20%): avoid misses by flagging anyone who might be sick. Extra follow-up tests are acceptable

Rule of thumb

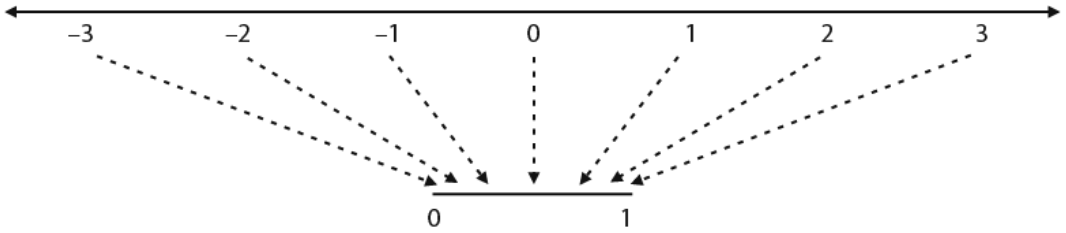
- Costly false positives (false alarms) → raise the threshold.
- Costly false negatives (misses) → lower it.

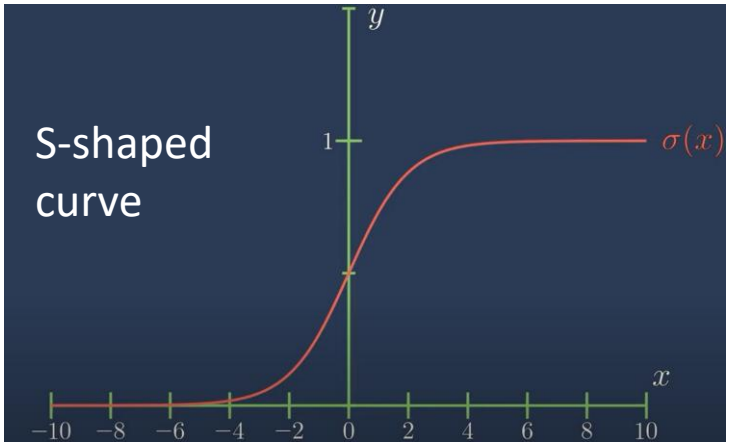
Sigmoid (Logistic) Function

- The **sigmoid (logistic) function** turns any number into a **probability between 0 and 1**
- It is written **σ (sigma)** and defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

  - **e** ≈ 2.718 is a fixed math constant
  - **Big positive z** → close to 1 (Yes); **big negative z** → close to 0 (No)





Logistic Regression - Model Representation

- Given **m** samples and **n** features, the model predicts the probability of the positive class, e.g. Pass.

Example: will a student pass?

- The model learns two things:
  - Weights **w** (feature importance)
  - Bias b

X dataset matrix	w parameters vector
$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix}$	$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}$

Where:

$X \in \mathbb{R}^{m \times n}$ input dataset matrix,
$m =$ number of samples,
$n =$ number of input features,
$\mathbf{w} \in \mathbb{R}^{n}$ (n)−dimensional weight vector,
$b \in \mathbb{R}$ bias term (scalar),
$\hat{\mathbf{y}} \in \mathbb{R}^{m}$ (m)−dimensional output vector

Step 1: Linear score

$$z = \hat{\mathbf{y}} = X\mathbf{w} + b$$

Step 2: Probability

- Squash score z into a probability with the **sigmoid function**:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- Output is always between **0 and 1**; predict Yes if ≥ 0.5

Learning a Logistic Regression Model

- Goal: learn **weights** $\mathbf{w} = [w_1, \ldots, w_m]$ and **bias** b that minimize the **Binary Cross-Entropy cost** (the model's error)

$$J = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(\hat{y}^{(i)}\right) + \left(1 - y^{(i)}\right)\log\left(1 - \hat{y}^{(i)}\right)\right]$$

  - $y^{(i)}$: actual label (0 or 1) of example i
  - $\hat{y}^{(i)}$: predicted probability that example i belongs to class 1

Intuition: how the cost behaves

- Low cost = predictions close to the true labels
- Confident wrong predictions are penalized heavily
- The model is pushed to predict:

  $\hat{y} \to 1$ when $y = 1$

  $\hat{y} \to 0$ when $y = 0$

Gradient Descent for Logistic Regression

Objective:
Find the best weights $\mathbf{w}$ and bias $\boldsymbol{b}$ (lowest cost)

Algorithm, step by step

1. **Initialize** weights $\mathbf{w}$ and bias $\boldsymbol{b}$ (e.g., zeros)

2. **Predict probabilities** with the current values:

$$\hat{y}^{(i)} = \sigma(\hat{\mathbf{y}} = X\mathbf{w} + b\ )$$

3. **Compute gradients** (which way lowers the cost) for $\mathbf{w}$ and $\boldsymbol{b}$

4. **Update parameters:** Update w and b by subtracting their gradients multiplied by the learning rate, moving toward a lower cost.

5. **Repeat** steps 2–4 until the cost stops decreasing (converged ✅)

In practice: scikit-learn runs this loop for you with `model.fit(X, y)`

Example: Will a Student Pass?

- **Goal:** predict whether a student passes (1) or fails (0)
- **Features (X):** hours_studied, attendance_pct
- **Target (y):** passed is a binary outcome (Yes/No), making logistic regression an appropriate choice
- **Data:** 200 students (synthetic, for illustration)

hours_studied	attendance_pct	passed
7.7	89	**1**
4.4	99	**0**
8.6	75	**1**
7.0	57	**1**
0.9	51	**0**

Reading the table

Each row = one student

Inputs (X) = first two columns

Label (y): 1 = pass, 0 = fail

The 5-step workflow



Implementing It with scikit-learn

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

df = pd.read_csv("students.csv")  # 1. load data
X = df[["hours_studied", "attendance_pct"]]
y = df["passed"]

X_train, X_test, y_train, y_test = train_test_split(  # 2. split train / test
    X, y, test_size=0.2, random_state=42, stratify=y)

model = LogisticRegression().fit(X_train, y_train)  # 3. fit the model
model.coef_, model.intercept_  # w = [0.809, 0.031]  b = -6.633
accuracy = model.score(X_test, y_test)  # 4. evaluate: 0.825

# 5. Predict for ONE student: 6 hours of study, 80% attendance
student = pd.DataFrame([[6, 80]], columns=X.columns)
print(model.predict_proba(student))  # [[0.336 0.664]]
# = [P(fail), P(pass)]: 2 classes, so 2 probabilities that add up to 1
```

82.5%

Accuracy: 33 of 40 unseen test students predicted correctly

66%

Chance this student (6 h, 80% attendance) passes. Fail = 34%

≈ 2.2×

Each extra study hour multiplies the odds of passing by e^0.809 ≈ 2.2

≈ 1.03×

Each extra 1% of attendance multiplies the odds by e^0.031 ≈ 1.03 (≈ 1.4× per +10%)

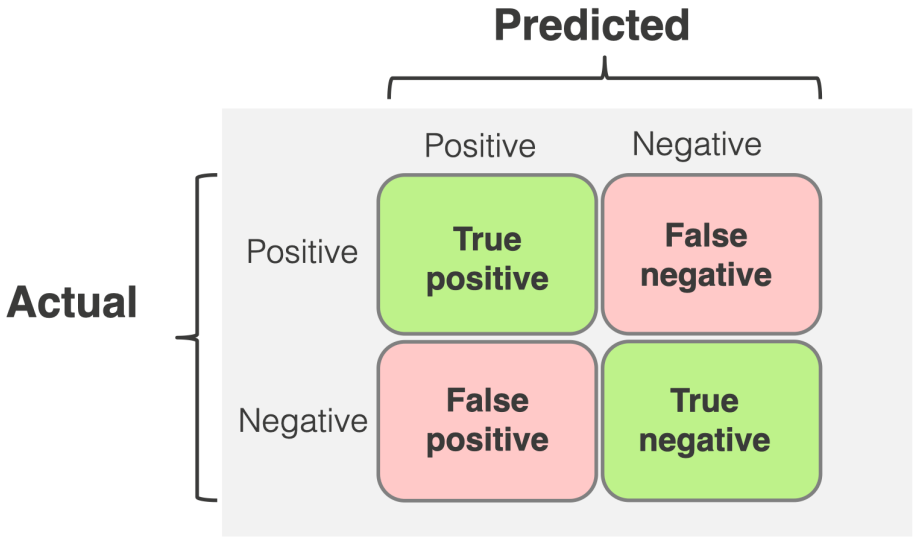
Classification Model Evaluation



Evaluating a Classifier

How good are the predictions?

- Compare predictions with the actual answers
- A confusion matrix counts the right and wrong ones
- Positive = what we look for (e.g., Cat)



✅ **True Positive (TP):** predicted Cat, actually Cat

✅ **True Negative (TN):** predicted Not Cat, actually Not Cat

❌ **False Positive (FP) = false alarm:** predicted Cat, actually Not Cat

❌ **False Negative (FN) = miss:** predicted Not Cat, actually Cat

💡 **Memory aid:** True/False = was the model right? Positive/Negative = what the model predicted.

Worked Example: Is It a Cat? 🐱



Cat	Not Cat	Cat	Not Cat	Cat	Not Cat	Cat	Not Cat	Cat	Cat

Labels above = what the model predicted. 7 of the 10 images are really cats.

	Predicted	
	Cat	**Not Cat**
Actual **Cat**	**TP = 5**	**FN = 2**
Actual **Not Cat**	**FP = 1**	**TN = 2**

7 of 10 predictions are correct. Next: turn these counts into metrics.

How to read the matrix

✅ **TP = 5:** real cats the model found

✅ **TN = 2:** dog and bird correctly rejected

❌ **FP = 1 (false alarm 🚨):** sheep called a cat

❌ **FN = 2 (misses ⚠️):** real cats called Not Cat

4 Key Metrics (Cat Example)

Metric	Question it answers	Formula	Cat example
Accuracy	How often is the model right overall?	`(TP + TN) / All`	(5 + 2) / 10 = 70%
Precision	When it says “Cat”, how often is it right?	`TP / (TP + FP)`	5 / (5 + 1) = 83%
Recall	Of all real cats, how many did it find?	`TP / (TP + FN)`	5 / (5 + 2) = 71%
F1 score	Are precision and recall both good? $\text{F1 Score} = \frac{2 \times (\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}}$	`2·P·R / (P + R)`	2·0.83·0.71 / (0.83 + 0.71) ≈ 0.77

Memory aid

Precision cares about false alarms (FP).

Recall cares about misses (FN).

Watch out: accuracy can mislead

If 95 of 100 images are not cats, a model that always says “Not Cat” is 95% accurate but finds 0 cats.

Which Metric Should I Use?

Precision

Use when false alarms (FP) are costly

Example: spam filter. Don’t block real emails.

Recall

Use when misses (FN) are costly

Example: disease screening. Don’t miss a sick patient.

F1 score

Use when both mistakes matter

Example: fraud detection, or when one class is rare.

Accuracy

Use only if classes are balanced

Both mistakes should cost about the same.

Rule of thumb: costly false alarms → raise the threshold. Costly misses → lower it.

Student Model: Confusion Matrix

```
from sklearn.metrics import confusion_matrix
y_pred = model.predict(X_test)   # 1 = pass, 0 = fail (threshold 0.5)
print(confusion_matrix(y_test, y_pred))  # [[19  3]
                                         #  [ 4 14]]
```

What happened to the 40 test students

	Predicted Fail	Predicted Pass
Actual Fail (22)	**19** True Negative	**3** False Positive
Actual Pass (18)	**4** False Negative	**14** True Positive

How to read it

Blue cells = correct. 19 + 14 = 33 of 40, so accuracy = 82.5%.

3 false alarms: failing students we predicted would pass.

4 misses: students who passed but we predicted fail.

Student Model: Precision & Recall

```
from sklearn.metrics import precision_score, recall_score, f1_score
precision_score(y_test, y_pred)   # 0.82
recall_score(y_test, y_pred)      # 0.78
f1_score(y_test, y_pred)          # 0.80
```

82%

Precision

`TP / (TP + FP)`
`= 14 / 17`

When we predict Pass, we are right 82% of the time.

78%

Recall

`TP / (TP + FN)`
`= 14 / 18`

We find 78% of the students who really pass.

0.80

F1 score

`2·P·R / (P + R)`

One number that balances precision and recall.

What if we lower the threshold to 0.4 instead of 0.5? Predict Pass when P(pass) ≥ 0.4

- Recall increases to 83% (15 of 18): We correctly identify more students who will pass.
- Precision drops to 71% (15 of 21): More students are incorrectly predicted to pass.

Key takeaway: Choose the threshold based on the **relative cost of mistakes**: missing a student who needs support (low recall) vs. flagging a student unnecessarily (low precision).